

» Newton's Method and Friends

- * So far we've focussed on gradient descent i.e. using gradient to provide direction of update.
- * But there are other ways to choose the descent direction. We already saw that with Adagrad which uses:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha [b_1 \frac{\partial f}{\partial x_1}(\mathbf{x}_t), b_2 \frac{\partial f}{\partial x_2}(\mathbf{x}_t), \dots, b_n \frac{\partial f}{\partial x_n}(\mathbf{x}_t)]$$

$$\text{with } b_i = \frac{1}{\sqrt{\sum_{t=1}^T \frac{\partial f}{\partial x_i}(\mathbf{x}_t)^2}}$$

→ when b_i are not all the same then descent direction is different from

$$[\frac{\partial f}{\partial x_1}(\mathbf{x}_t), \frac{\partial f}{\partial x_2}(\mathbf{x}_t), \dots, \frac{\partial f}{\partial x_n}(\mathbf{x}_t)]$$

- * An older, much more widely used approach for choosing the b_i 's is Newton's method, and its various variants → its a core method in optimisation, although not a big data method

» First-order Local Approximation

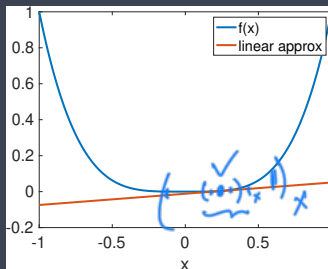
- * Suppose we have function $f(x)$ with x scalar
- * Recall

$$f(x) \approx f(x') + \frac{df}{dx}(x')(x - x')$$

i.e. we can *locally* approximate $f(\cdot)$ with a linear function.

Approximation is only accurate when $|x - x'|$ is small

- * E.g. $f(x) = x^4$, $df/dx = 4x^3$, $x' = 0.25$:



- * Choosing $x = x' - \alpha \frac{df}{dx}$ then moving from point x' to x tends to decrease function $f(\cdot)$ *if step size is small*
→ need small step size because linear approx is only accurate when $|x - x'|$ is small

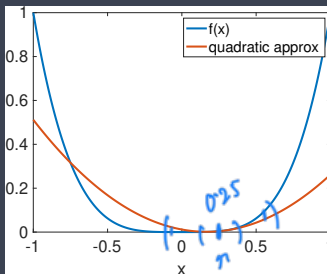
» Second-order Local Approximation

- * We can also use other local approximations to $f(x)$, obvious choice is a quadratic:

$$f(x) \approx f(x') + \frac{df}{dx}(x')(x - x') + \frac{1}{2} \frac{d^2f}{dx^2}(x')(x - x')^2$$

$$\frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x} \left(\frac{\partial f}{\partial x} \right)$$

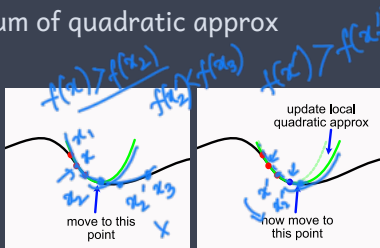
- * $\frac{d^2f}{dx^2}(x)$ is the derivative of $\frac{df}{dx}(x) \rightarrow \frac{df}{dx}(x)$ is just a function of x , so its ok to take its derivative
- * $\frac{df}{dx}$ is called the *first derivative* and $\frac{d^2f}{dx^2}$ the *second derivative*
- * E.g. $f(x) = x^4$, $df/dx = 4x^3$, $d^2f/dx^2 = 12x^2$, $x' = 0.25$:



- * How to use this approx to take a descent step?

» Newton's Method

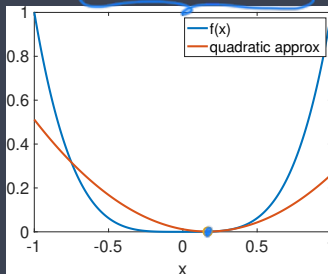
- * Fit a local quadratic approximation/model to function around current point x
- * Move to minimum of quadratic approx
- * Repeat



- * Also sometimes called a *sequential quadratic programme (SQP)* since make repeated quadratic approximations to function $f(\cdot)$ and find min of each of these individual *quadratic programmes*

» Second-order Local Approximation (Optional)

- * Quadratic approx $f(x) \approx f(x') + \frac{df}{dx}(x')(x - x') + \frac{1}{2} \frac{d^2f}{dx^2}(x')(x - x')^2$



- * Quadratic approx is minimised by:

$$x = x' - \frac{df/dx(x')}{d^2x/df^2(x')}$$

- * E.g. $f(x) = x^4$, $df/dx = 4x^3$, $d^2f/dx^2 = 12x^2$, $x' = 0.25$:

$$x = 0.25 - \frac{4 \times 0.25^3}{12 \times 0.25^2} = 0.25 - 0.0833 = 0.1667$$

(marked by 'o' on plot above)

» Second-order Local Approximation (Optional)

Finding min of $f(x) \approx f(x') + \frac{df}{dx}(x')(x - x') + \frac{1}{2} \frac{d^2 f}{dx^2}(x')(x - x')^2$:

* At minimum derivative is zero i.e.

$$\frac{d}{dx} \left(f(x') + \frac{df}{dx}(x')(x - x') + \frac{1}{2} \frac{d^2 f}{dx^2}(x')(x - x')^2 \right) = 0$$

i.e.

$$\frac{df}{dx}(x') + \frac{d^2 f}{dx^2}(x')(x - x') = 0$$

i.e.

$$\rightarrow x - x' = - \frac{df/dx(x')}{d^2 f/dx^2(x')} \quad \checkmark$$

» When x Is A Vector (Optional)

- * In general, when vector x as n elements then linear approx is:

$$f(x) \approx f(x') + \frac{\partial f}{\partial x_1}(x')(x_1 - x'_1) + \frac{\partial f}{\partial x_2}(x')(x_2 - x'_2) + \dots + \frac{\partial f}{\partial x_n}(x')(x_n - x'_n)$$

- * and quadratic approx is:

$$\begin{aligned} f(x) \approx & f(x') + \underbrace{\frac{\partial f}{\partial x_1}(x')(x_1 - x'_1) + \frac{\partial f}{\partial x_2}(x')(x_2 - x'_2) + \dots + \frac{\partial f}{\partial x_n}(x')(x_n - x'_n)}_{\text{Linear approx}} \\ & + \frac{\partial^2 f}{\partial x_1^2}(x')(x_1 - x'_1)^2 + \frac{\partial^2 f}{\partial x_1 \partial x_2}(x')(x_1 - x'_1)(x_2 - x'_2) + \dots + \frac{\partial^2 f}{\partial x_1 \partial x_n}(x')(x_1 - x'_1)(x_n - x'_n) \\ & + \frac{\partial^2 f}{\partial x_n \partial x_1}(x')(x_n - x'_n)(x_1 - x'_1) + \frac{\partial^2 f}{\partial x_n \partial x_2}(x')(x_n - x'_n)(x_2 - x'_2) + \dots + \frac{\partial^2 f}{\partial x_n \partial x_n}(x')(x_n - x'_n)^2 \end{aligned}$$

Handwritten notes: Blue arrows point to the linear and quadratic terms. Blue squiggly lines are on the left. Blue handwritten terms $\frac{\partial^2 f}{\partial x_1 \partial x_2}$, $\frac{\partial^2 f}{\partial x_2^2}$, and $\frac{\partial^2 f}{\partial x_2 \partial x_n}$ are added below the corresponding terms in the quadratic approximation.

- * Here $\frac{\partial^2 f}{\partial x_1 \partial x_2}$ is the partial derivative of $\frac{\partial f}{\partial x_1}$ wrt x_2 etc
- * *Can use e.g. sympy to calculate these partial derivatives* ✓
- * *But can see that calculating quadratic approx is much more complicated (so computationally expensive) than calculating linear approx → quadratic approx not really suitable for large problems*

» When x Is A Vector (Optional)

- * Vector/matrix notation:

$$\left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

$$\nabla f(x') = \left[\frac{\partial f}{\partial x_1}(x'), \frac{\partial f}{\partial x_2}(x'), \dots, \frac{\partial f}{\partial x_n}(x') \right]$$

$$\nabla^2 f(x') = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2}(x') & \frac{\partial^2 f}{\partial x_1 \partial x_2}(x') & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n}(x') \\ \frac{\partial^2 f}{\partial x_2 \partial x_1}(x') & \frac{\partial^2 f}{\partial x_2^2}(x') & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n}(x') \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1}(x') & \frac{\partial^2 f}{\partial x_n \partial x_2}(x') & \dots & \frac{\partial^2 f}{\partial x_n^2}(x') \end{bmatrix}$$

and

$$f(x) \approx f(x') + \nabla f(x')(x - x') + \frac{1}{2}(x - x')^T \nabla^2 f(x')(x - x')$$

- * Matrix $\nabla^2 f$ is called the *Hessian* of f

- * Quadratic approx is minimised by choosing:

$$x = x' - (\nabla^2 f(x'))^{-1} \nabla f(x')$$

This is *Newton step*. Here $(\nabla^2 f(x'))^{-1}$ is the *inverse* of matrix $\nabla^2 f(x')$ i.e. a matrix such that

$$(\nabla^2 f(x'))^{-1} \nabla^2 f(x') = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix} = I$$

» Newton's Method¹

- * Iterative minimisation algorithm is:

→ $t = 0$

Repeat:

→ $x_{t+1} = x_t - \alpha (\nabla^2 f(x_t))^{-1} \nabla f(x_t)$
 $t = t + 1$

inverse of Hesse;

grad.

where α is step size and $\nabla^2 f$ is the Hessian of f .

- * Matrix notation can be off-putting, what's really going on here?

- * Rewrite update as

→ $x_{t+1} = x_t - \alpha [b_1 \frac{\partial f}{\partial x_1}(x_t), b_2 \frac{\partial f}{\partial x_2}(x_t), \dots, b_n \frac{\partial f}{\partial x_n}(x_t)]$

- * When $b_1 = b_2 = \dots = b_n = 1$ have usual gradient descent update

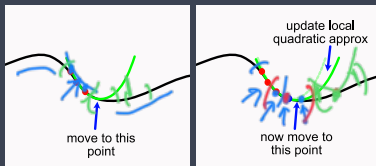
- * *Newton's method uses the curvature of $f(\cdot)$ to select weights $b_1, b_2, \dots, b_n$ i.e. selects them to minimise local quadratic approx to $f(\cdot)$.*

- * Compare with Adagrad which uses $b_i = \frac{1}{\sqrt{\sum_{t=1}^T \frac{\partial f}{\partial x_i}(x_t)^2}}$

¹https://en.wikipedia.org/wiki/Newton's_method_in_optimization

» Trust Regions

- * Newton step is quite aggressive, can cause update to diverge → common to combine Newton update with a *trust region* approach
- * *Trust region*: restrict the size of the move made. Adapt size of trust region based on progress e.g.
 - * If at next point x_{t+1} the cost function decrease is similar to prediction from quadratic model then *increase trust region/step-size*
 - * If cost function is v different from expected (e.g. function increases rather than decreases) then *decrease the trust region/step-size*



$$|x - x'| \leq \epsilon$$
$$\epsilon = 0.001$$

- * *Can also just use a line search of course.*

» Computational Cost

- * Storing the Hessian can take a lot of memory e.g. if have 1M parameters $\nabla^2 f$ is a $1M \times 1M$ matrix i.e. with 10^{12} elements (that's 8000GB with 64bit floats)
- * Computing the Hessian and its inverse can be expensive
- * Lots of neat tricks used to save memory and reduce computation, e.g. see https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.newton_krylov.html
- * Also common to bypass calculation of exact Hessian and instead calculate a cheaper approximation → *quasi-Newton methods*
- * Again, lots of clever approximations. One popular one is used in BFGS algorithm², or the more memory-efficient L-BFGS variant.
 - * L-BFGS is available in pytorch³, BFGS in tensorflow⁴
 - * L-BFGS is default solver in `sklearn.linear_model.LogisticRegression`

²https://en.wikipedia.org/wiki/Broyden-Fletcher-Goldfarb-Shanno_algorithm

³<https://pytorch.org/docs/stable/generated/torch.optim.LBFGS.html>

⁴https://www.tensorflow.org/probability/api_docs/python/tfp/optimizer/bfgs_minimize

https://www.tensorflow.org/probability/api_docs/python/tfp/optimizer/bfgs_minimize

» Summary

- * Newton and quasi-Newton methods are second-order methods → locally approximate function with a quadratic function and then minimise that, then repeat
- * Popular, tend to converge in few iterations.
- * But relatively high memory and computation cost →
 - * Not suited to big data or models with large number of parameters (e.g. deep neural nets)
 - * But often a good choice for smaller optimisation problems
- * Combining Newton method with SGD not well studied (now noisy Hessian as well as noisy gradient), and remember regularisation effect of SGD noise for deep neural nets ... which is also poorly understood but likely important